



Global Digital Bank (GDB)

A Hands On Case Study for System Design

Complexity: 6 / 10 (Medium to Hard) | **Implementation Medium:** Java / Python

Document Version: 2.0 | **Date:** October 26, 2023 | **Status:** Final | **Prepared By:** GDB Architecture & Product Team |

Classification: Confidential

Table of Contents

01

Executive Summary

03

Project Architecture & Core Principles

05

Data Rules & Models

5.1. Core Data Rules 5.2. Data Model Specification 5.3. Storage Rules & Persistence 5.4. Security & Compliance Rules 5.5. Error Handling Rules

07

User Interface & Workflows

7.1. CLI Menu Structure 7.2. Sample Workflows

02

Purpose and Strategic Goals

04

Functional Requirements

4.1. Base Features (Core Implementation) 4.2. Extended Features (Full Implementation) 4.3. Agentic AI Features (LangChain/LangGraph Implementation)

06

Non-Functional Requirements (NFRs)

6.1. Performance & Scalability 6.2. Security & Compliance 6.3. Reliability & Resilience 6.4. Maintainability & Extensibility 6.5. Data Migration & Portability

08

Development & Deliverables

8.1. Development Workflow 8.2. Deliverables

1. Executive Summary

The Global Digital Bank (GDB) project is an advanced, multi-layered digital banking platform designed to transform the financial experience for customers of Bank of Success Private Limited. GDB provides a comprehensive suite of features for account management, transactions, analytics, and administrative control. The platform is architected to empower customers with secure, accessible, and intelligent banking services.

This document defines the complete scope of the GDB project as an **individual implementation**. The developer will build a robust Python-based command-line banking system that evolves from a simple application into a sophisticated platform incorporating all 30+ features, including cutting-edge Agentic AI capabilities using LangChain and LangGraph.

The project serves as a comprehensive demonstration of software engineering excellence, including Object-Oriented Design (OOP), modular architecture, file persistence, error handling, and advanced AI integration. The final product will be a holistic, high-performance banking system that showcases innovation, technical depth, and production-grade quality.

 Industry insight: According to McKinsey, digital banking platforms that integrate AI-driven features see up to 30% improvement in customer satisfaction scores and 20% reduction in operational costs.

2. Purpose and Strategic Goals

Purpose: The purpose of GDB is to create a secure, efficient, and intelligent digital banking platform that meets the evolving needs of modern customers and the bank's operational requirements. Key purposes include:



Intelligent Customer Empowerment

Provide a platform where users can manage their finances and receive intelligent, proactive insights.



Streamlined Operations

Automate and simplify account management, transaction processing, and data management.



Secure & Compliant Infrastructure

Build a system with robust security measures, ensuring data protection and compliance with regulatory standards.



Adaptable & Scalable Architecture

Design a modular system that can easily integrate new features, third-party services, and scale to handle a growing user base and transaction volume.

Strategic Goals

1 Comprehensive Feature Coverage

Implement all 30+ features (Base, Extended, and AI) in a single, cohesive application.

2 User-Centric Banking

Deliver an intuitive and feature-rich experience through a well-designed CLI.

3 Intelligent Automation

Integrate Agentic AI to provide predictive analytics and proactive financial management.

4 Real-Time Transaction Processing

Support immediate processing with high throughput and low latency.

5 Robust Architecture

Implement a layered, maintainable, and scalable design.

6 Secure Data Management

Ensure the highest level of data security and compliance.

7 Performance Excellence

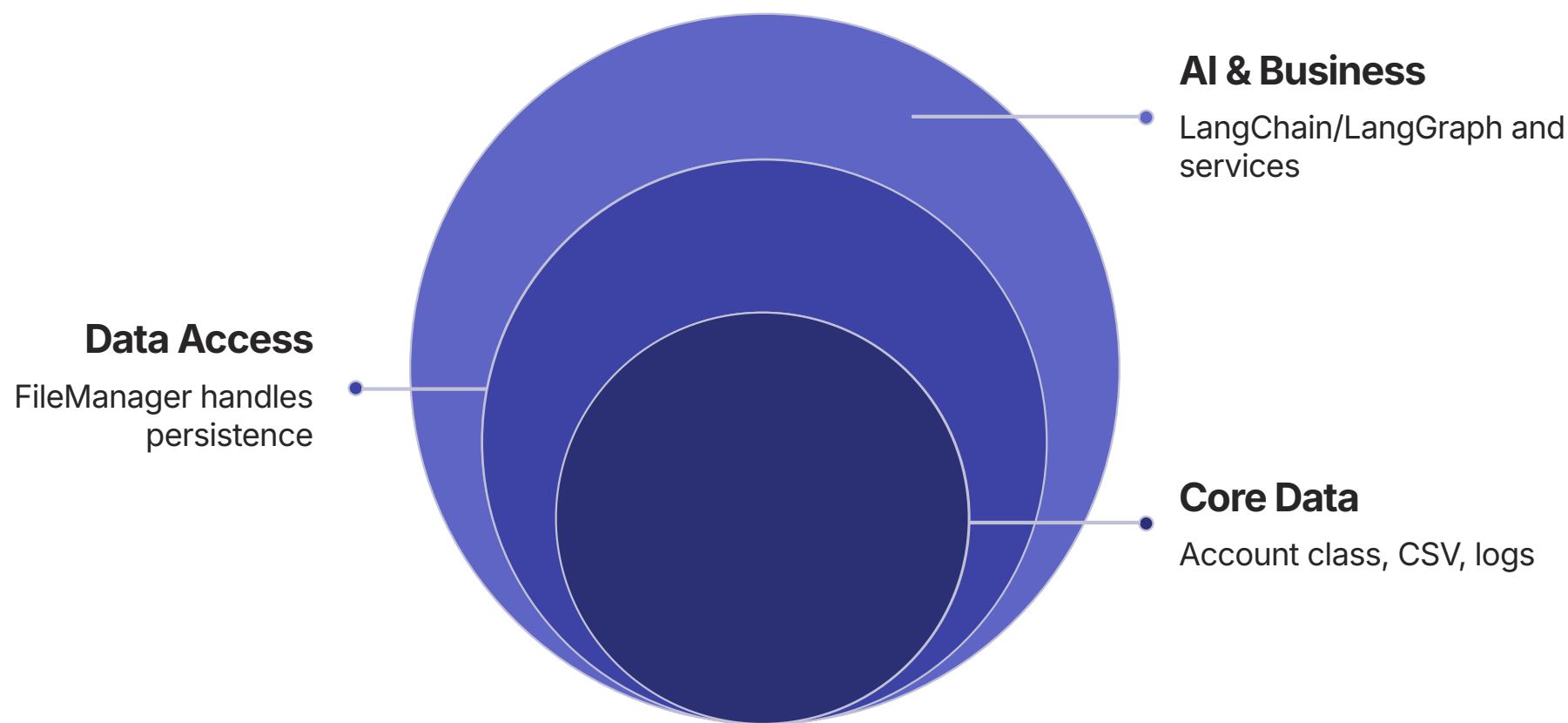
Meet strict SLAs for response times, concurrency, and transaction rates.

8 Production-Ready Quality

Produce well-tested, documented, and maintainable code.

3. Project Architecture & Core Principles

The GDB will adhere to a modular, multi-layered architecture, separating concerns to ensure maintainability, scalability, and testability.



Presentation Layer (Controller) A menu-driven Command-Line Interface (CLI) that handles user interaction. This layer parses user input and delegates requests to the Services Layer.	Business Logic Layer (Services) Contains the core application logic (e.g., BankingService, AnalyticsService). It handles validation, rule enforcement, and orchestrates transactions. It interacts with the Data Access Layer for persistence.	AI/Automation Layer A separate, modular layer that leverages LangChain/LangGraph to implement Agentic AI features. This layer interacts with the Business Logic Layer to perform intelligent operations.
Data Access Layer Responsible for reading and writing data to files (e.g., FileManager). It handles the specifics of CSV and log file operations with robust error handling and atomic operations.	Data Layer Includes the Account class and the physical data stores (accounts.csv, transactions.log).	

Core Principles

Single Responsibility

Each class and module has a single, well-defined purpose.

Separation of Concerns

The UI, business logic, and data storage are decoupled.

Open/Closed Principle

The system should be open for extension (adding new features) but closed for modification (existing code should not break).

Dependency Injection

Dependencies are injected rather than hard-coded to improve testability.

 Industry insight: Systems built on SOLID principles reduce bug introduction rates by up to 40% during feature additions, according to IEEE Software Engineering studies.

4. Functional Requirements

The system will include three categories of features: Base, Extended, and Agentic AI. All features will be implemented by a single developer.

4.1. Base Features (Core Implementation)

These features form the foundational code of the banking system.

Feature ID	Feature Name	Description
B1	Create Account	Create new accounts with name, age, type, and auto-generated account number. Reject age < 18.
B2	Deposit	Add funds to an account with all applicable rules.
B3	Withdraw	Deduct funds from an account if sufficient balance and rules are met.
B4	Balance Inquiry	Display account balance and details.
B5	Close Account	Mark an account as inactive.
B6	File Persistence	Save/load account data to/from accounts.csv.

4.2. Extended Features — Account Management Module

The developer will implement all 24 extended features across all modules.

Feature ID	Feature Name	Description
E1	Search by Name	Search and display accounts matching a given name string (case-insensitive, partial matches allowed).
E2	Search by Account #	Search and display a specific account by its unique account number.
E3	List All Active Accts	Display all accounts with status "Active" in a formatted table.
E4	List All Closed Accts	Display all accounts with status "Inactive/Closed" in a formatted table.
E5	Reopen Closed Account	Change an account's status from "Inactive" to "Active" after PIN verification.
E6	Rename Account Holder	Update the name of an account holder after PIN verification.
E7	Delete All Accounts	A secure function to delete all account data (requires confirmation and admin-level access).
E8	Count Active Accounts	Display the total number of active accounts.

4.2. Extended Features — Transactions, Analytics & More

Transactions & Rules Module

Feature ID	Feature Name	Description
E9	Transaction Log File	Write all transactions (with timestamps) to transactions.log in the specified format.
E10	Transaction History	Display a chronological list of all transactions for a specific account with pagination.
E11	Minimum Balance Check	Enforce a minimum balance of ₹500 for Savings and ₹1000 for Current accounts on every withdrawal/transfer.
E12	Daily Transaction Limit	Limit total withdrawals per account to ₹50,000 per day. Track daily withdrawal totals in memory and persist.
E13	Transfer Funds	Transfer funds between two distinct, active accounts. Logs both a debit and a credit transaction.

Analytics Module

Feature ID	Feature Name	Description
E14	Top N Accounts by Balance	Display the top N accounts with the highest balances in descending order.
E15	Average Balance Calculator	Calculate and display the average balance across all active accounts.
E16	Youngest Account Holder	Find and display the account holder with the minimum age.
E17	Oldest Account Holder	Find and display the account holder with the maximum age.
E18	Simple Interest Calculator	Calculate simple interest for a given account (or a generic calculation) with customizable rate and time.

File Operations, Security & System Modules

Feature ID	Feature Name	Description
E19	Export Accounts to Report	Export a formatted report of all accounts (or a filtered list by type/status) to a new file (e.g., report.txt).
E20	Import Accounts from File	Batch create accounts by reading data from a structured file (e.g., import.csv) with proper validation.
E21	PIN/Password Protection	Implement a 4-digit numeric PIN per account. Required for sensitive operations (Withdraw, Transfer, Close, Reopen).
E22	System Exit with Autosave	Gracefully shut down the system, automatically saving all in-memory data to persistent storage.

4.3. Agentic AI Features

This section introduces a sophisticated layer of "Agentic AI" capabilities, making GDB an intelligent banking assistant. All features will be built using LangChain and LangGraph.

A1

Intelligent Fraud Detection Agent

A2

Automated Financial Advisor (Smart Savings)

A3

Natural Language Query Interface (NLQI)

A4

Predictive Analytics Dashboard



Industry context: The global cost of financial fraud exceeded \$485 billion in 2023 (Nasdaq Global Financial Crime Report). AI-powered fraud detection reduces false positives by up to 60% compared to rule-only systems.

Feature A1: Intelligent Fraud Detection Agent

Description

A LangGraph-powered agent that monitors transaction patterns in real-time. It acts by analyzing transaction amount, frequency, and account history against a set of rules and learned patterns.

Capabilities

- Real-time monitoring of all deposit, withdrawal, and transfer operations.
- Rule-based detection (e.g., transfer > ₹50,000 to a new account, multiple large withdrawals in a short period).
- Anomaly detection using statistical methods (e.g., transactions exceeding 3 standard deviations from the account's average).

Actions

If a transaction is flagged as suspicious, the agent can:

- **Log** the suspicious event with detailed metadata.
- **Block** the transaction and return an error to the user.
- **Request** additional authentication (simulated via a prompt).
- **Notify** a simulated admin via a log message.

Implementation

Uses a StateGraph to define a loop: Agent State → Analyze → Decide → Log/Block/Notify → Agent State.

Feature A2: Automated Financial Advisor (Smart Savings)

Description

A LangChain agent that analyzes account data to provide proactive financial advice and automated savings actions.

Capabilities

- Analyzes transaction history (last 30 days) to identify spending patterns and categories.
- Calculates potential savings by identifying discretionary spending (e.g., dining, entertainment).
- Provides personalized recommendations: "Based on your spending, you could save ₹5,000/month by reducing dining out."
- Implements automated savings rules (e.g., "Round up every debit transaction to the nearest ₹100 and transfer the difference to a Savings account.").

Actions

- **Analyze** transaction history using LangChain tool calls.
- **Recommend** a savings plan based on analysis.
- **Execute** automated transfers if the user approves.

Implementation

Uses a LangChain agent with tools to access transaction history, account balances, and perform transfers.

Industry Insight

Personalised AI financial advice increases customer savings rates by an average of 18%, per Deloitte Digital Banking Survey 2023.

Feature A3: Natural Language Query Interface (NLQI)

A feature that allows users to interact with the bank using natural language prompts, abstracting the complexity of the CLI.

Capabilities

- Understands and responds to a wide range of banking queries.
- Handles complex requests like "Show me all my transactions from last month" or "What is the total amount I spent on groceries in the last 30 days?"
- Executes actions like "Transfer ₹5,000 to account 1005."

Examples

Query 1

User: "What is my account balance for account 1001?"

Query 2

User: "Show me my last 10 transactions."

Query 3

User: "Transfer ₹200 to account 1002."

Query 4

User: "How much did I spend on food this month?"

Implementation: Uses LangChain's LLMChain with a carefully crafted prompt template that includes the available tool functions, their descriptions, and the account context. The LLM translates natural language into structured function calls.

Feature A4: Predictive Analytics Dashboard

Description

An AI agent that pre-calculates and caches key performance metrics to power an admin dashboard.

Capabilities

- Predicts the probability of an account becoming dormant (no transactions in 6 months).
- Forecasts cash flow for the bank based on past transaction trends.
- Identifies high-value customers (top 5% by balance) and potential churn risks.
- Generates a comprehensive summary report.

Implementation

Uses LangChain to generate a summary report by analyzing raw transaction and account data. This report is presented as a formatted text output to an admin user.

Why It Matters

Banks using predictive analytics report a 25% reduction in customer churn and a 15% increase in cross-sell revenue (Accenture Banking Technology Vision 2023).

5. Data Rules & Models

5.1. Core Data Rules — Account Creation

Age Restriction Customer must be ≥ 18 years.	Account Number Auto-generated, unique, and non-duplicate (starting from 1000).
Account Types Allowed types are "Savings" and "Current" only.	Minimum Balance on Creation Savings: Minimum initial deposit of ₹500. Current: Minimum initial deposit of ₹1,000.

Transaction Rules

Deposit

- Amount must be positive (> 0).
- Single deposit limit: ₹100,000.

Withdrawal

- Amount must be positive (> 0).
- Post-withdrawal balance must not fall below the minimum allowed: Savings: $\geq$ ₹500. Current: $\geq$ ₹1,000.
- Insufficient funds $\rightarrow$ transaction rejected.

Daily Limit (E12)

- Total withdrawals per account per day $\leq$ ₹50,000.
- Resets at midnight (based on system time).

Transfer (E13)

- Both accounts must exist and be active.
- Minimum balance rule for the sender is strictly enforced.
- Transfer is logged as two transactions: debit (sender) and credit (receiver).

Account Status Rules

- **Active:** Default status on creation.
- **Inactive (Closed):** Operations (deposit, withdraw, transfer) are disallowed.
- **Reopen (E5):** Only allowed via the "Reopen Account" feature after PIN verification.

5.2. Data Model Specification

Account Class

- account_number (int, unique, auto-generated)
- name (str, non-empty)
- age (int, positive, ≥ 18)
- balance (float, $\geq$ minimum balance for the account type)
- account_type (str, "Savings" or "Current")
- status (str, "Active" or "Inactive")
- pin (int, optional, 4-digit numeric for protection)
- daily_withdrawal_total (float, resets daily)
- last_withdrawal_date (datetime, for tracking daily limit)

Transaction Log Entry

- timestamp (datetime)
- account_number (int)
- operation (str: "Create", "Deposit", "Withdraw", "Transfer", "Close", "Reopen", etc.)
- amount (float, if applicable)
- balance_after (float)

Scheduled Transfer (for AI Feature)

- schedule_id (int, unique)
- source_account (int)
- target_account (int)
- amount (float)
- frequency (str: "One-time", "Daily", "Weekly", "Monthly")
- next_execution_date (datetime)

5.3. Storage Rules & Persistence

1 Persistence

Account data is persisted to `accounts.csv` on every successful operation. Transaction logs are appended to `transactions.log` on every operation.

2 File Format

accounts.csv format:

`account_number,name,age,balance,type,status,pin,daily_withdrawal_total,last_withdrawal_date`

transactions.log format:

`timestamp,account_number,operation,amount,balance_after`

3 Data Integrity

File write operations must be atomic. Use a temporary file and rename strategy to prevent corruption in case of a system crash.

4 Backup

The system should automatically create a backup of `accounts.csv` (e.g., `accounts_backup.csv`) before performing any destructive write operation.

5.4. Security & Compliance Rules

PIN Authentication (E21)

The 4-digit PIN must be entered correctly for sensitive operations (Withdraw, Transfer, Close Account, Reopen Account).

Active Account Check

Operations are only allowed on accounts with the "Active" status.

Data Privacy

Customer data (name, account number) should not be exposed unnecessarily in logs or error messages. Use generic error messages (e.g., "Invalid credentials" instead of "Account not found").

Input Sanitisation

All user inputs must be sanitised to prevent injection attacks (e.g., malicious CSV data).

Audit Trail

All critical actions (PIN changes, account closures, admin actions) must be logged.



Industry context: 95% of cybersecurity breaches are caused by human error (IBM Security Report 2023). Robust input sanitisation and audit trails are the first line of defence.

5.5. Error Handling Rules

Error Scenario	Handling Rule
Invalid Input	Use try/except to catch non-numeric inputs, type mismatches, and display friendly messages. The system must never crash due to user input.
Invalid Values	Reject negative values for deposit, withdrawal, and age with descriptive messages.
Account Not Found	Display "Account not found. Please check the account number and try again."
Inactive Account	Disallow any operation on an inactive account with a message: "Account is inactive. Please reopen the account or contact support."
Insufficient Balance	Display a clear message: "Insufficient balance. Minimum balance requirement: ₹XXX."
File Errors	Gracefully handle file not found or permission errors by creating default structures and logging the error.

6. Non-Functional Requirements (NFRs)

The following NFRs are critical for the GDB project to be considered a high-quality, production-ready system.

6.1. Performance & Scalability

500+

TPS

Transactions per second in a single-threaded Python environment, with optimised file I/O.

<100ms

Read Latency

Average response time for 99% of read requests (e.g., Balance Inquiry).

<250ms

Write Latency

Average response time for 99% of write requests (e.g., Deposit, Withdraw, Transfer).

10K+

Accounts

The system should efficiently handle 10,000+ accounts without significant performance degradation.

1M+

Log Entries

The system should efficiently handle 1,000,000+ transaction logs without significant performance degradation.

<500MB

RAM

The in-memory data structure should be optimised to handle the maximum data volume within <500MB of RAM.

Concurrency: The architecture must be designed to support concurrent access (e.g., using file locking or a database) for future multi-user scenarios.

Scalability: The design should be easily scalable to support a database backend (e.g., PostgreSQL) in the future with minimal code changes.

Response Time (User Experience): The maximum time between a user's input and the display of a result must be < 2 seconds for complex operations (e.g., generating a report) and < 500ms for simple operations (e.g., balance inquiry).



Industry benchmark: Top-tier digital banks like Revolut process over 150 million transactions per month. Latency under 250ms is the industry standard for maintaining customer trust in digital banking.

6.2. Security & Compliance

Security

Adherence to the Principle of Least Privilege. Users and system components should only have the minimum access required.

Compliance

The system must be designed to be compliant with relevant regulations (e.g., GDPR, PCI-DSS) by design. This includes data encryption at rest and in transit, comprehensive audit trails, and secure user authentication.

Data Encryption

Sensitive data (PII, financial information) should be encrypted at rest (AES-256). A simple encryption utility can be implemented for accounts.csv.

Authentication

Support multi-factor authentication (MFA) for all sensitive user actions and administrative access. A basic second factor (e.g., a one-time code) can be simulated.

Authorisation

Implement role-based access control (RBAC). Roles: Customer (standard user), Admin (full access). Different menus for each role.

Audit Trail

Every action that creates, modifies, or deletes data, or is performed by an administrator, must be logged with the user ID and timestamp. Logs must be immutable.

Session Management

For future web/mobile versions, design with session management in mind.



Industry context: The average cost of a data breach in the financial sector reached \$5.9 million in 2023 (IBM Cost of a Data Breach Report). GDPR and PCI-DSS non-compliance fines can reach €20 million or 4% of global annual turnover.

6.3. Reliability & Resilience

Data Integrity & Atomicity

All transactions must adhere to the ACID properties (Atomicity, Consistency, Isolation, Durability). If any part of a multi-step transaction fails, the entire transaction must be rolled back.

Availability

Target a system uptime of **99.9%** ("Three Nines") for the current CLI application.

Disaster Recovery

There must be a clearly defined disaster recovery (DR) plan with a Recovery Point Objective (RPO) of **< 1 hour** and a Recovery Time Objective (RTO) of **< 4 hours**. At a minimum, this means regular backups of data files.

Fault Tolerance

The system must be fault-tolerant to individual component failures (e.g., file system errors). It should automatically detect and recover from failures using retry mechanisms and graceful degradation.

Graceful Shutdown (E22)

The "System Exit" feature must guarantee 100% data persistence, even in the event of an unexpected interruption (e.g., via signals like SIGTERM). Use signal handlers to ensure data is saved.

- 📘 Industry context: 99.9% uptime ("Three Nines") equates to only ~8.7 hours of downtime per year. Leading banks target 99.99% ("Four Nines") — just 52 minutes of downtime annually. Every minute of banking downtime costs an average of \$5,600 (Gartner).

6.4. Maintainability & Extensibility

Modularity

The system must be built with a modular architecture, as described in Section 3, allowing individual components to be updated or replaced without affecting the entire system. Ease of extension is a primary design goal.

Code Quality

The codebase must have a minimum test coverage of **80%** (unit and integration tests). It should follow PEP 8 style guidelines and be well-documented.

Documentation

Comprehensive docstrings for all classes and functions. A detailed README with setup, usage, and architecture overview.

CI/CD Integration

Code must be written to be compatible with Continuous Integration/Continuous Deployment (CI/CD) pipelines for automated testing and deployment. Include a requirements.txt file.

Configurability

Use a configuration file (e.g., config.py or .env) for constants like minimum balances, daily limits, and file paths.

Why 80% Test Coverage?

Google's engineering research shows that projects with >80% test coverage have 40% fewer production bugs. Teams with CI/CD pipelines deploy 208x more frequently and recover from incidents 2,604x faster (DORA State of DevOps Report 2023).

6.5. Data Migration & Portability

→ Data Migration

Provide a utility to migrate account and transaction data from legacy systems (e.g., flat files from accounts.csv) into the current data store.

→ Backward Compatibility

The system's data reading/writing logic must be backward compatible to support future migrations.

→ Export/Import

The file operations (E19, E20) must be robust and handle large datasets efficiently, possibly with batch processing and progress indicators.

→ Versioning

Implement a data file versioning system to handle schema changes gracefully.

7. User Interface & Workflows

7.1. CLI Menu Structure

The CLI will be role-based, showing different menus for customers and administrators.

Customer Menu

```
=====
=====
Welcome to Global Digital Bank
1) Create Account
2) Deposit
3) Withdraw
4) Balance Inquiry
5) Close Account
6) Search by Name
7) Search by Account Number
8) List All Active Accounts
9) List All Closed Accounts
10) Reopen Closed Account
11) Rename Account Holder
12) Transaction History
13) Transfer Funds
14) Top N Accounts by Balance
15) Average Balance Calculator
16) Youngest Account Holder
17) Oldest Account Holder
18) Simple Interest Calculator
19) AI: Financial Advisor (Smart Savings)
20) AI: Natural Language Query
21) System Exit with Autosave
Enter your choice:
```

Admin Menu (Additional Options)

```
22) Delete All Accounts (Admin Only)
23) Export Accounts to Report (Admin Only)
24) Import Accounts from File (Admin Only)
25) AI: Predictive Dashboard (Admin Only)
```

7.2. Sample Workflows

1. Standard Workflow

01	02	03
Create Account User selects "Create Account". System prompts for Name, Age, and Account Type. System validates input and creates the account, showing "Account created successfully! Account Number: 1001".	Deposit User selects "Deposit". System prompts for Account Number and Amount. User enters "1001" and "500". System processes deposit, logs the transaction, and shows "Deposit successful. New balance: ₹500".	System Exit User selects "System Exit". The system saves all data and displays "Goodbye!".

2. Agentic AI Workflow (Natural Language Query)

01	02	03
Select NLQI User selects "AI: Natural Language Query". System prompts: "💡 Ask me anything about your account:"	Balance Query User enters: "What is my balance for account 1001?" The LangChain agent parses the query, executes the get_balance("1001") function, and returns "✅ The balance for account number 1001 is ₹500."	Transfer Command User enters: "Transfer ₹200 to account 1002." The agent processes the command, performs the transfer("1001", "1002", 200) function, and returns "✅ Successfully transferred ₹200 from account 1001 to account 1002."

3. Agentic AI Workflow (Fraud Detection)

01	02	03
Large Transfer Attempt User attempts a large transfer: "Transfer ₹60,000 to account 1003." The Fraud Detection Agent intercepts the transaction.	Flagged & Challenged The agent analyses the transfer pattern and flags it as suspicious (amount exceeds daily limit or account's normal behaviour). The agent responds: "⚠️ This transaction has been flagged for security review. Please verify your identity by entering your PIN."	Verified & Logged Upon successful PIN verification, the transaction proceeds. The agent logs the event: "Suspicious transaction flagged and approved after PIN verification."

8. Development & Deliverables

8.1. Development Workflow



8.2. Deliverables

1 Source Code

A single zip file containing all Python files, modularised and well-documented.

2 Data Files

Sample accounts.csv and transactions.log files generated by the application.

3 requirements.txt

A file listing all Python dependencies (e.g., langchain, langgraph, pytest).

4 README.md

A detailed file with clear instructions on how to set up, run, and test the application. It must include: Setup instructions (installing dependencies). How to run the application. Architecture overview. Complete feature list with descriptions. How to test the AI features (with sample prompts).

5 Test Report

A report showing unit test coverage (must be $\geq 80\%$) and the results of integration tests.

6 User Manual

A simple guide for end-users explaining how to use all features, including the AI capabilities.